RECURSO DE PROFUNDIZACIÓN – CLASE 9

Back-end moderno y APIs básicas con Node.js y Express

Propósito del recurso

Este recurso tiene como finalidad ofrecer al estudiante una comprensión más profunda y conceptual de los temas abordados en la Clase 9. Aquí se explican, de forma ampliada, los elementos fundamentales del desarrollo backend, el rol de las APIs REST y la arquitectura de proyectos en Node.js con Express. Se incluyen analogías, diagramas, flujos lógicos y explicaciones reforzadas para garantizar que el estudiante entienda intuitiva y técnicamente cómo funciona el backend moderno.

1. Comprender Node.js desde una perspectiva moderna

Node.js no es un lenguaje ni un framework; es un entorno de ejecución basado en el motor V8 (el mismo que usa Google Chrome), capaz de ejecutar JavaScript fuera del navegador. Su mayor potencial está en su capacidad de manejar muchas solicitudes simultáneamente sin bloquearse.

1.1 ¿Por qué Node.js es tan veloz?

Node usa un mecanismo llamado event loop, un bucle que recibe tareas y las resuelve sin bloquear el sistema. Mientras un lenguaje tradicional procesa cada operación esperando a que termine (bloqueo), Node deriva operaciones lentas (como consultas a disco o BD) a hilos auxiliares y continúa recibiendo solicitudes.

1.2 Analogía: Node como un restaurante rápido

- En un restaurante tradicional (PHP, Python sin async), el mesero atiende una mesa a la vez.
- En Node.js, el mesero solo toma pedidos rápidamente, los envía a la cocina y mientras la comida se cocina, atiende otras mesas.

Esta filosofía hace que Node sea ideal para APIs y microservicios.

2. ¿Qué es Express y por qué se usa tanto?

Express es un framework minimalista para crear servidores HTTP y APIs REST.
No impone una estructura; solo ofrece las piezas fundamentales:

- Crear rutas
- Manejar parámetros
- Consumir JSON
- Implementar middleware
- Enviar respuestas HTTP

Por ello es perfecto para aprendizaje y para proyectos reales que luego evolucionarán a arquitecturas más complejas.

2.1 Vista general del funcionamiento de Express

Aquí un diagrama que resume la lógica básica de Express:



3. Arquitectura del proyecto: separación de responsabilidades

Uno de los objetivos conceptuales más importantes de la clase es que el estudiante entienda por qué se separa el backend en carpetas como routes, controllers y models.

3.1 Comparación

Componente	**Equivalente en la vida real**	**Función**
Rutas	Recepcionista	Dice a qué área dirigir la solicitud
Controladores	Encargado del área	Toma decisiones y coordina la operación
Modelos	Sistema interno	Accede a los datos reales y devuelve información
Middleware	Guardias/ filtros	Verifican permisos, anotan logs o inspeccionan datos

Así el estudiante entiende que un backend profesional nunca mezcla todo en un solo archivo porque debe ser mantenible y escalable.

4. CRUD: el corazón de cualquier API

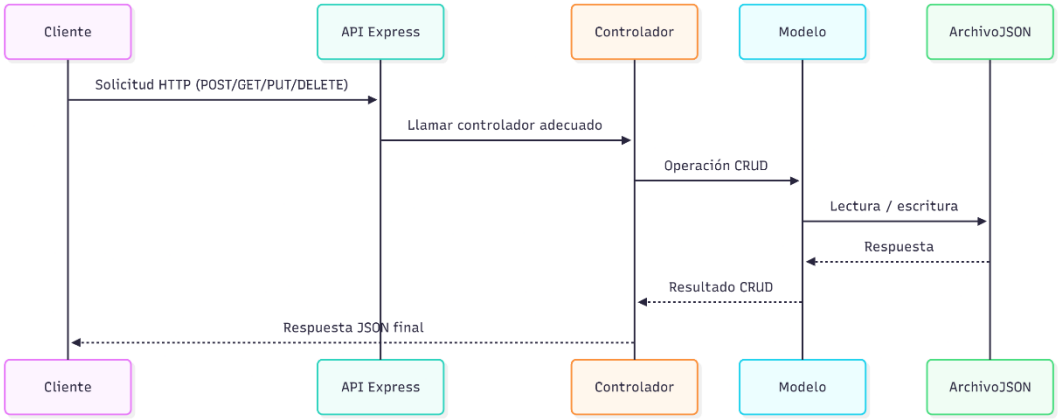
CRUD significa:

- **C**reate
- **R**ead
- **U**pdate
- **D**elete

Y corresponde directamente a los verbos HTTP:

CRUD	**Método HTTP**	**Ejemplo**
Create	POST	POST /api/productos
Read (all)	GET	GET /api/productos
Read (one)	GET	GET /api/productos/:id
Update	PUT	PUT /api/productos/:id
Delete	DELETE	DELETE /api/productos/:id

4.1 Diagrama conceptual del flujo CRUD



5. Middlewares: el "pasillo" obligatorio antes de entrar a cada ruta

Un middleware es una función que intercepta la petición antes de llegar al controlador.

Los casos más típicos:

- **logger** → registra hora, método, URL.
- **auth** → verifica si hay usuario autenticado.
- **validación** → revisa que el JSON entrante sea válido.
- **errorHandler** → captura errores globales.

5.1 Flujo conceptual de middleware



6. Manejo de errores: la importancia de no "romper" la API

Cuando ocurre un error en un backend:

- no se debe cerrar la aplicación
- no se debe enviar un HTML al cliente
- no se deben filtrar errores internos

Por eso la clase usa el middleware errorHandler.

Ejemplo de mala práctica:
❌ enviar errores sin estandarizar.

Ejemplo de buena práctica:
✓ devolver siempre JSON consistente:

```
{
  "status": "error",
  "message": "Error interno del servidor",
  "detail": "Archivo no encontrado"
}
```

Esto prepara al estudiante para API profesionales.

7. Buenas prácticas esenciales para APIs REST

1. Endpoints centrados en recursos

Correcto:

- /api/productos
- /api/productos/15

Incorrecto:

- /api/crearProducto
- /api/leeLista

2. Métodos HTTP con semántica correcta

- GET no modifica nada
- POST crea
- PUT actualiza completamente
- DELETE elimina

3. Respuestas JSON estandarizadas

Siempre incluir:

- status
- data o message

4. No enviar datos sensibles

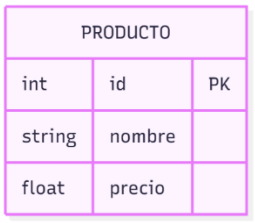
Nunca enviar contraseñas, tokens incorrectos o rutas internas.

5. Modificar solo lo necesario

Especialmente importante cuando pasemos a bases de datos reales.

8. Representación ER conceptual (para preparar la semana 11)

Aunque hoy usamos JSON, el modelo conceptual es equivalente al de una BD real.



El propósito es que el estudiante visualice que:
lo que hoy es JSON mañana será SQL o MongoDB.

9. Conclusión del recurso

Este recurso de profundización complementa la Clase 9 al ofrecer una visión más conceptual del backend moderno con Node.js y Express. Se aclaran los roles de las rutas,

controladores, modelos y middlewares, así como la importancia del CRUD y del manejo adecuado de errores.

La comprensión sólida de estos conceptos permitirá que el estudiante avance sin dificultades hacia la Semana 11, donde se conectará el backend con bases de datos reales, ORMs y consultas estructuradas.